RedPyme Gestión

Sistemas Microinformáticos y Redes

Marcos Garcia, Iker Carrizal y Marcos Padilla

Sara Pulgar Pantoja

13/5/26

IES EL CAÑAVERAL

Departamento de Informática

Memoria del proyecto

Documento técnico

Índice

1.- Introducción 1
1.1.- Descripción y contexto del proyecto 2
1.2.- Motivación del proyecto 2
1.3.- Beneficios esperados 2
2.- Objetivo/s generales del proyecto 2
3.- Contexto actual 2
4.- Análisis de la necesidad 2
5.- Diseño de la solución 2
5.1.- Lógico. 3
5.2.- Físico. 3
6.- Desarrollo de la solución 3
6.1.- Tecnologías y herramientas utilizadas (lenguajes de programación, frameworks, etc.). 3
7.- Planificación del proyecto 3
7.1.- Acciones 3
7.2.- Temporalización y secuenciación 4
8.- Pruebas y validación 4
9.- Relación del proyecto con los módulos del ciclo 4
10.- Conclusiones 5
11.- Proyectos futuros 6
12.- Bibliografía/Webgrafía 6
13.- Anexos 6

1.- Introducción

1.1.- Descripción y contexto del proyecto

NetManager es un panel de control basado en web diseñado para simplificar la administración de servidores Ubuntu en entornos PYME. Se centra en la gestión de servicios críticos de red como DHCP y FTP sin necesidad de usar la terminal.

1.2.- Motivación del proyecto

Resolver la dificultad que tienen las pequeñas empresas sin personal técnico para configurar servicios de red complejos, ofreciendo una solución intuitiva y viable.

1.3.- Beneficios esperados

Automatización de tareas, reducción de errores de configuración manual y despliegue rápido me-diante una imagen ISO personalizada.

2.- Objetivo/s generales del proyecto

Desarrollar una herramienta integral de administración de sistemas Linux que permita gestionar servicios de red de forma visual y segura.

3.- Contexto actual

Estado del arte.-

Actualmente existen paneles como Webmin o Cockpit, pero NetManager busca una experiencia más ligera, personalizada y enfocada en la facilidad de instalación mediante una imagen de sistema propia.

Conceptos clave.-

Flask (backend), Subprocess (ejecución de comandos), DHCP (protocolo de red), vsftpd (servidor FTP)

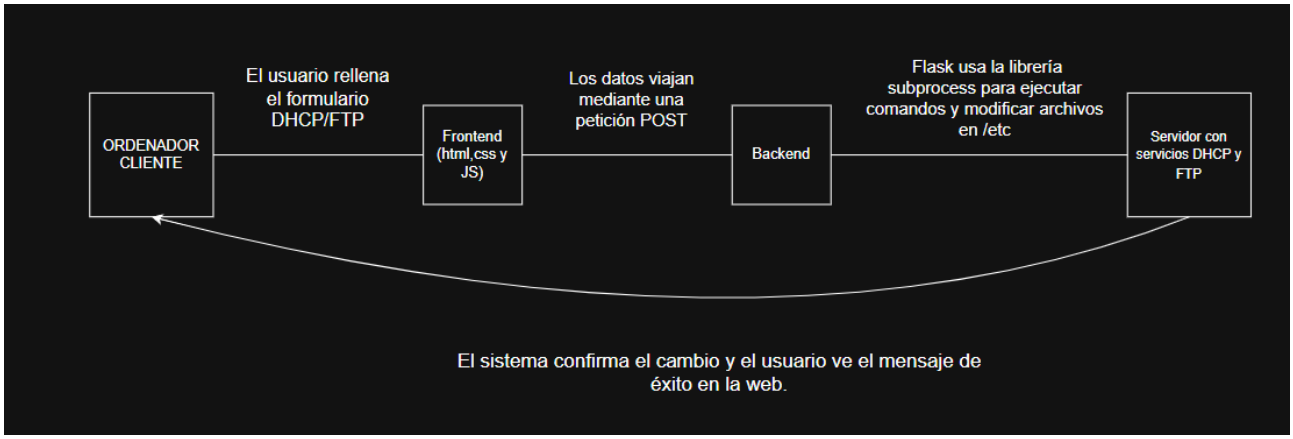
4.- Análisis de la necesidad

El proyecto cubre la necesidad de gestionar de forma dinámica y visual servicios que habitualmente requieren edición manual de archivos de texto en /etc , lo cual es propenso a fallos humanos.

5.- Diseño de la solución

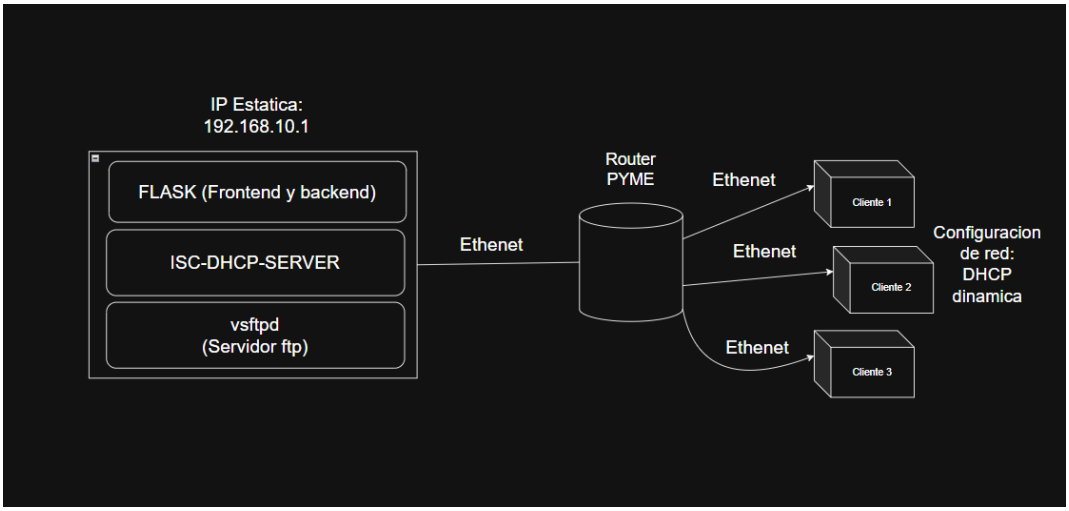
Este apartado tendrá las decisiones que se han tomado para la realización del proyecto:

5.1.- Lógico.



5.2.- Físico.

Esquema de la arquitectura del sistema.



6.- Desarrollo de la solución

Pasos de implementación detallados.

6.1.- Tecnologías y herramientas utilizadas (lenguajes de programación, frameworks, etc.).

- **Python/Flask:** Elegido por su ligereza y facilidad para interactuar con scripts de sistema.
- **Visual Studio Code:** Utilizado como entorno de desarrollo principal por su modularidad y extensiones de acceso remoto SSH.
- **VirtualBox:** Infraestructura para el testeo de la ISO y las máquinas cliente.
- **Vsftpd:** Servicio base para FTP por su robustez y seguridad

6.2 Gestión de FTP (vsftpd)

- **Instalación (I):** Instalación del paquete vsftpd mediante el gestor de paquetes apt.
- **Configuración (C):** Ajuste del archivo /etc/vsftpd.conf para habilitar escrituras y enjaular usuarios (chroot). Configuración de privilegios en visudo para permitir que Flask gestione usuarios.
- **Operación (O):** Uso del formulario web para crear usuarios y asignar contraseñas automáticamente mediante scripts de Python.
- **Mantenimiento (M):** Monitorización del servicio con systemctl y revisión de logs de acceso.

6.2 Gestión de DHCP (isc-dhcp-server)

- **(I) Instalación:** Despliegue del servicio DHCP en la ISO.
- **(C) Configuración:** Lógica de Python para modificar el archivo /etc/dhcp/dhcpd.conf respetando la sintaxis de red.
- **(O) Operación:** Introducción de rangos de IP y máscaras desde el panel de control.
- **(M) Mantenimiento:** Verificación de concesiones activas.

7.- Planificación del proyecto

7.1.- Acciones

Para abordar el proyecto, se habrá dividido en varios bloques de acciones o tareas complejas, y a su vez en una serie de tareas más sencillas.

Por ejemplo, si seguimos las fases principales de desarrollo de un proyecto serían:

División en hitos:
1) Análisis y diseño
2) Core en Flask y DHCP
3) Servicio FTP y pulido de UI.

7.2.- Temporalización y secuenciación

Mes	Semana	Tareas	Hito al que pertenece cada tarea
Marzo	S1	**Inicio:** Reunión de equipo, definición de nuestro proyecto, ISO Appliance.	Hito 1
	S2	**Análisis:** Reparto de módulos (Iker, Marcos G., Padilla), diseño lógico/físico.	Hito 1
	S3	**Entorno:** Instalación de Ubuntu Server focal y VS Code.	Hito 1
	S4	**Entrega Hito 1:** Documentación inicial, creación de preguntas/respuestas.	**Hito 1 (26/03)**
Abril	S1	**Core Flask:** Programación de la estructura del app.py, rutas básicas.	Hito 2
	S2	**Módulo DHCP:** Lógica de subprocess para /etc/dhcpd.conf, permisos sudoers.	Hito 2
	S3	**UI Hito 2:** Diseño de la primera versión del HTML/CSS e integración.	Hito 2
	S4	**Entrega Hito 2:** Grabación del vídeo (musitado), pulido técnico final.	**Hito 2 (20/04)**
Mayo	S1	**Módulo FTP:** Instalación/configuración de vsftpd, gestión de usuarios.	Hito 3
	S2	**Depuración Extrema:** Resolución del DDoS en repositorios de Ubuntu, DNS.	Hito 3
	S3	**Revisión: Comprobación de que todo funciona correctamente**	Hito 3
	S4	**Entrega Hito 3:** Vídeo final (con voces), Memoria Completa (ICOM).	**Hito 3 (14/05)**

8.- Pruebas y validación

Para cada servicio crítico gestionado por NetManager, se han ejecutado las siguientes pruebas a través de la interfaz web:

Servicio / Tarea a Validar	Prueba Realizada	Resultado Espera-do	Resultado Real	Estado
Módulo Core (Flask)	Acceso a la IP del servidor.	Carga de la Dashboard.	Dashboard carga rápido.	**OK**
DHCP	Guardar nueva subred.	Archivo dhcpd.conf actualizado.	Archivo actualiza-do con datos.	**OK**
DHCP (Valida-ción Real)	Arrancar cliente (VM) en la red.	Asignación de IP dinámica.	Cliente recibe IP del rango.	**OK**
FTP (vsftpd)	Crear usuario test_user.	Usuario añadido en Ubuntu.	Usuario creado sin shell.	**OK**

9.- Relación del proyecto con los módulos del ciclo

SISTEMAS OPERATIVOS EN RED

- **Competencia aplicada:** Administración avanzada de servidores Linux.
- **Justificación técnica:** NetManager se ha desarrollado sobre un servidor Ubuntu Server 22.04 LTS (Jammy). El proyecto ha requerido la configuración de permisos avanzados mediante el archivo /etc/sudoers para que el backend (Flask) ejecute comandos de sistema (useradd, chpasswd, systemctl), y la gestión de usuarios y grupos locales para el servicio FTP.

SERVICIOS EN RED

- **Competencia aplicada:** Despliegue y configuración de servicios de red.
- **Justificación técnica:** El núcleo del proyecto es la automatización de protocolos críticos. Se han implementado scripts para la configuración dinámica de /etc/dhcpd.conf (ISC-DHCP-Server) y la administración de /etc/vsftpd.conf (vsftpd). Además, se ha diseñado el Appliance para que resuelva DNS de forma autónoma, garantizando la operatividad del panel incluso en escenarios de contingencia de red.

APLICACIONES WEB

- **Competencia aplicada:** Desarrollo y despliegue de soluciones web Full Stack.
- **Justificación técnica:** Se ha desarrollado una interfaz completa (Frontend en HTML5/CSS3) que se comunica con un servidor de aplicaciones (Backend en Python con Flask). Se ha programado la lógica para capturar peticiones HTTP (peticiones POST de formularios) y traducirlas a acciones de sistema, implementando un panel de control totalmente funcional y dinámico.

SEGURIDAD Y ALTA DISPONIBILIDAD

- **Competencia aplicada:** Implementación de medidas de seguridad y preconfiguración ante fallos.
- **Justificación técnica:** El proyecto aborda la seguridad en múltiples niveles: el panel web implementa control de acceso, el servicio FTP usa chroot para aislar a los usuarios en sus directorios, y la concepción de Appliance offline mitiga riesgos externos como los ataques DDoS a repositorios globales (un punto clave validado en las pruebas de contingencia).

10.- Conclusiones

Tras terminar el proyecto NetManager PYME, hemos alcanzado los objetivos planteados inicialmente. El desarrollo de este Appliance ha permitido no solo demostrar nuestras capacidades técnicas en la administración de sistemas, sino también la viabilidad de crear herramientas que humanicen y simplifiquen la gestión de redes complejas.

Las principales conclusiones extraídas de este proyecto son:

1. **Automatización como valor añadido:** Se ha comprobado que la abstracción de la línea de comandos mediante una interfaz Flask reduce drásticamente el error humano. Tareas que antes requerían saber modificar archivos de configuración ahora se realizan de forma segura y visual.
2. **Resolución de contingencias:** Durante el proyecto hemos tenido varios problemas, como la caída de repositorios globales de Ubuntu o fallos de resolución DNS y hemos aprendido a ser más resilientes, diagnosticando y tomando una solución más robusta y autónoma (Appliance offline).
3. **Seguridad y Control:** Hemos aprendido la importancia de gestionar correctamente los privilegios de sistema. El uso de sudoers para que el servidor web interactúe con el sistema operativo se ha realizado bajo criterios de mínimos privilegios, garantizando que el servidor sea seguro y estable.
4. **Trabajo en equipo:** El desarrollo de NetManager nos ha ayudado a entender como se comportan diferentes elementos entre sí y como hacen sinergia (Sistemas, Backend y Diseño UI), pareciéndose a un entorno de trabajo real donde la integración de cada módulo es obligatoria para el éxito del producto final.

En resumen, NetManager se presenta como una solución funcional y profesional para PYMES, cumpliendo con la idea de que la tecnología más avanzada es esa que es invisible y fácil de usar para el usuario final.

11.- Proyectos futuros

NetManager ha sido diseñado con una arquitectura modular que permite la expansión de sus funcionalidades. Debido a la priorización de la estabilidad del servicio FTP y el cumplimiento de los plazos del Hito 3, se han identificado las siguientes líneas de desarrollo para futuras versiones:

- **Implementación de Servidor Web (Apache):** La evolución más natural del proyecto es integrar la gestión de Virtual Hosts. Esto permitiría a las empresas no solo almacenar archivos vía FTP, sino también hostear sus propias páginas web o intranets directamente desde el panel, automatizando la creación de archivos de configuración
- **Estadísticas de Uso y Monitorización:** Integrar gráficas de consumo de CPU, memoria y tráfico de red en tiempo real en el panel principal.
- **Copias de Seguridad Automatizadas:** Crear una sección donde el usuario pueda programar backups de las carpetas de los usuarios FTP hacia un almacenamiento externo o la nube.

12.- Bibliografía/Webgrafía

Canonical Ltd. (2026), "Documentación oficial de Ubuntu Server 22.04 LTS (Jammy Jellyfish)", https://ubuntu.com/server/docs

Pallets Projects (2024), "Flask Documentation (v3.0.x)", https://flask.palletsprojects.com/

Python Software Foundation (2026), "Subprocess management — Python 3.10 documentation", https://docs.python.org/3/library/subprocess.html

vsftpd Project (2023), "Man pages: vsftpd.conf configuration file", https://security.appspot.com/vsftpd/vsftpd_conf.html

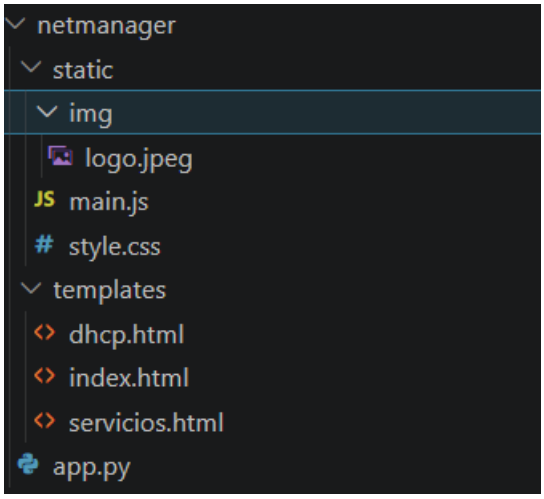
ISC (Internet Systems Consortium), "DHCP Documentation and User Guides", https://www.isc.org/dhcp/

13.- Anexos

Anexo I: Estructura del Proyecto (Directorios)

A continuación, se detalla la jerarquía de archivos del servidor Flask. Esta estructura permite la separación entre la lógica de control (Python) y la interfaz visual (HTML/CSS).

- **app.py**: Archivo principal con las rutas y lógica de subprocess.
- **templates**: Contiene los archivos HTML (diseñados por Padilla).
- **static**: Contiene los archivos CSS, imágenes y logos de la empresa.



Anexo II: Fragmentos de Código Críticos (Logic Backend)

Se incluye a continuación el bloque de código principal del archivo app.py. Este fragmento es el encargado de recibir los datos del formulario web y sobrescribir el archivo de configuración del servidor DHCP de ISC (/etc/dhcp/dhcpd.conf).

```
@app.route('/configurar/dhcp', methods=['GET', 'POST'])
def configurar_dhcp():
    if request.method == 'GET':
        #Formulario
        return render_template('dhcp.html')

    elif request.method == 'POST':
        # Recogida dats
        subred = request.form['subred']
        mascara = request.form['mascara']
        rango_inicio = request.form['rango_inicio']
        rango_fin = request.form['rango_fin']
        router = request.form['router']

        # Creamos configuración para archivo de servidor
        nueva_configuracion = f"""
# Generado por NetManager
default-lease-time 600;
max-lease-time 7200;
authoritative;

subnet {subred} netmask {mascara} {{
    range {rango_inicio} {rango_fin};
    option routers {router};
    option domain-name-servers 8.8.8.8, 8.8.4.4;
}}
"""
```